SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	AmirthaSriN	**Reg. No.:**	1911260175
Date:	27.10.2026	**Duration:**	60 minutes

Code of Conduct

I _AmirthaSri(1911260175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:
Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

1. Study the given problem statement and identify the system objectives.
2. Analyze the functional and non-functional requirements that influence the software architecture.
3. Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

1. Describe the purpose and responsibilities of each architectural component.
2. Explain how the components communicate and exchange data.
3. Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement	Thoroughly analyzes the problem	Analyzes most requirements with	Identifies only basic	Requirement analysis is

Analysis (30%)	statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	minor omissions or limited explanation.	requirements with partial analysis.	incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope;	Justification is weak or absent; report lacks organization and technical clarity.

	presents a well-organized, professional report.		report needs improvement.	

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

ARCHITECTURE DESIGN ANALYSIS

AI-Based College FAQ and Student Support Chatbot

1. System Requirements Analysis

The AI-Based College FAQ and Student Support Chatbot is designed to provide students with quick and reliable answers to common college-related questions. Students can ask about admission procedures, course details, class timings, examination schedules, fees, attendance rules, campus facilities, and important announcements. The main objective is to reduce the time spent waiting for responses from college offices and to make student support available through a simple digital interface.

Functional requirements describe what the system must do. The chatbot should accept questions in a chat interface, identify the user's intent, search the available college FAQ or knowledge base, and return a relevant answer. It should support common student queries, display contact details or direct students to the appropriate department when an answer is unavailable, and allow authorized staff to update FAQ information. The system may also keep basic conversation records for improving responses, while protecting students' personal information.

Non-functional requirements describe how well the system should operate. The chatbot should be easy to use, provide responses quickly, remain available when students need help, and handle multiple users at the same time. It should protect information through authentication and access control, provide reliable answers from approved college information, and be easy to maintain when rules, schedules, or announcements change.

The important quality attributes are scalability, maintainability, reliability, availability, performance, and security. Scalability allows the system to support more students as usage increases. Maintainability makes FAQ updates and software changes easier. Reliability helps the chatbot return consistent information. Availability keeps the service accessible, while performance supports quick responses. Security protects administrative access and any student information handled by the system.

2. Selection of Suitable Software Architecture

A layered architecture with a client-server approach is suitable for the AI-Based College FAQ and Student Support Chatbot. The client-server approach allows students to access the chatbot through a web or mobile interface, while the server processes their requests and retrieves answers. The layered structure separates the presentation layer, application or chatbot logic layer, AI and knowledge retrieval layer, and data layer.

This architecture is suitable because each layer has a clear responsibility. The user interface can be improved without changing the database, and the FAQ storage can be updated without redesigning the entire application. The architecture is also appropriate for a college project because it is understandable, cost-effective, and can be expanded later. If usage becomes very large, selected functions can be separated into independent services in the future.

3. Software Architecture Description

The proposed system contains the following major components: student interface, request handling module, chatbot and intent-processing module, FAQ and knowledge-base search module, database, response module, and administrator interface. These components work together to receive a question, find suitable information, and display an answer.

The student interface is the point through which students type questions and read responses. The request handling module receives the message and passes it to the chatbot logic. The chatbot module identifies the likely purpose of the question and prepares a search request. The knowledge-base module searches approved college FAQs and related information. The database stores FAQs, category details, and administrative updates. The response module formats the retrieved information into a clear answer and sends it back to the student. The administrator interface allows authorized college staff to add, edit, or remove FAQ entries.

4. Components and Their Interactions

When a student enters a question, the interface sends it to the application server. The request handling module validates the request and forwards the message to the chatbot module. The chatbot module analyses the question and identifies its topic, such as examinations, fees, attendance, or campus facilities. The knowledge-base module then searches the stored FAQ content for a matching answer.

If a relevant answer is found, the response module returns it to the student in a readable format. If the system cannot find a reliable answer, it should clearly say that the information is

unavailable and provide the relevant college office or support contact where possible. This prevents the chatbot from presenting an uncertain answer as a confirmed fact.

Authorized administrators can use the administration interface to update information. The updates are saved in the database and become available to future searches. Access controls should ensure that ordinary students can ask questions but cannot change official college information. The overall workflow is therefore: student question, request processing, intent analysis, FAQ search, answer preparation, and response display.

5. Quality Attribute Analysis

Scalability: The server-side application can be deployed on infrastructure that can be upgraded as the number of users increases. Frequently requested FAQ data can be optimized for faster retrieval. The system should be designed so that additional capacity can be added when student traffic grows.

Security: Administrative functions should require authorized login. Role-based access control should restrict who can change official FAQ content. Data should be transmitted securely, and the system should avoid collecting unnecessary personal information. Conversation records, if stored, should be protected and retained only when required.

Performance: Common questions should be answered quickly by searching a structured FAQ database. Clear categorization, indexing, and efficient queries can reduce response time. The interface should show a useful response without unnecessary delays.

Reliability: Answers should be based on verified college information. The system should handle invalid questions and temporary errors gracefully. If no matching answer is available, it should provide a clear fallback message rather than inventing information.

Maintainability: Separating the user interface, chatbot logic, and data storage makes the system easier to test and update. College staff can revise FAQ content through the administrator interface without requiring major changes to the application.

Availability: The chatbot should be hosted on a dependable server and monitored for errors. Regular backups of the FAQ database and a simple recovery process can help restore service if a failure occurs. Planned maintenance should be managed to reduce disruption to students.

6. Justification of Architectural Decisions

The layered client-server architecture is selected because it offers a clear separation of responsibilities and is suitable for a college FAQ system. It is simpler to develop and maintain than a fully distributed microservices architecture, which may add unnecessary complexity for an initial version. The architecture also supports future improvements because the interface, chatbot processing, and database can be modified with limited impact on other parts.

One trade-off is that a single application server may become a bottleneck if the number of users grows significantly. This can be addressed by increasing server capacity, optimizing database queries, and later separating high-demand functions into independent services. Another trade-off

is that chatbot answers depend on the quality and freshness of the college knowledge base. Therefore, authorized staff should review and update information regularly.

In the future, the system can be enhanced with multilingual support, voice input, integration with the college website or student portal, feedback ratings, and escalation to a human support team. Integration with official systems should use secure APIs and appropriate permissions. These enhancements can be introduced gradually without replacing the complete architecture.

Conclusion

The AI-Based College FAQ and Student Support Chatbot can make college information easier for students to access and reduce repetitive enquiries handled by staff. A layered client-server architecture provides a practical structure for processing questions, searching verified FAQs, managing content, and delivering responses. By focusing on security, performance, reliability, availability, scalability, and maintainability, the system can serve current needs and support future enhancements.